

code of Priority scheduling non-pre-emptive

```
#include <stdio.h>

int main()
{
    int n;
    printf("riya gupta ,usn:1WA24CS235 \n");

    printf("Enter number of processes: ");
    scanf("%d",&n);

    int pid[n], at[n], bt[n], pr[n];
    int ct[n], tat[n], wt[n];
    int completed[n];

    for(int i=0;i<n;i++)
        completed[i] = 0;

    // Input process details
    for(int i=0;i<n;i++)
    {
        printf("\nEnter details for Process %d\n",i+1);

        pid[i] = i+1;

        printf("Arrival Time: ");
        scanf("%d",&at[i]);

        printf("Burst Time: ");
        scanf("%d",&bt[i]);

        printf("Priority (Lower number = Higher priority): ");
        scanf("%d",&pr[i]);
    }

    int current_time = 0;
    int completed_count = 0;

    while(completed_count < n)
    {
        int highest_priority = 9999;
        int selected = -1;
```

```

for(int i=0;i<n;i++)
{
    if(at[i] <= current_time && completed[i] == 0)
    {
        if(pr[i] < highest_priority)
        {
            highest_priority = pr[i];
            selected = i;
        }
    }
}

if(selected == -1)
{
    current_time++;
}
else
{
    ct[selected] = current_time + bt[selected];

    tat[selected] = ct[selected] - at[selected];

    wt[selected] = tat[selected] - bt[selected];

    current_time = ct[selected];

    completed[selected] = 1;
    completed_count++;
}
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(int i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
}

return 0;
}

```

Output

```
riya gupta ,usn:1WA24CS235
Enter number of processes: 5

Enter details for Process 1
Arrival Time: 0
Burst Time: 3
Priority (Lower number = Higher priority): 5

Enter details for Process 2
Arrival Time: 2
Burst Time: 2
Priority (Lower number = Higher priority): 3

Enter details for Process 3
Arrival Time: 3
Burst Time: 5
Priority (Lower number = Higher priority): 2

Enter details for Process 4
Arrival Time: 4
Burst Time: 4
Priority (Lower number = Higher priority): 4

Enter details for Process 5
Arrival Time: 6
Burst Time: 1
Priority (Lower number = Higher priority): 1

PID AT  BT  PR  CT  TAT WT
P1  0   3   5   3   3   0
P2  2   2   3  11   9   7
P3  3   5   2   8   5   0
P4  4   4   4  15  11   7
P5  6   1   1   9   3   2
```

code of Priority scheduling Pre-emptive

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("riya gupta ,usn:1WA24CS235 \n");
```

```
    int n, i;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int pid[n], at[n], bt[n], pr[n];
```

```
    int rt[n], ct[n], tat[n], wt[n];
```

```
    // Input
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        printf("\nProcess %d\n", i + 1);
```

```
        printf("Process ID: ");
```

```
        scanf("%d", &pid[i]);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &bt[i]);
```

```
        printf("Priority: ");
```

```
        scanf("%d", &pr[i]);
```

```
        rt[i] = bt[i]; // Remaining time
```

```
    }
```

```
    int current_time = 0;
```

```
    int completed = 0;
```

```
    while(completed < n)
```

```
    {
```

```
        int highest_priority = 9999;
```

```
        int selected_process = -1;
```

```

// Find highest priority process
for(i = 0; i < n; i++)
{
    if(at[i] <= current_time && rt[i] > 0)
    {
        if(pr[i] < highest_priority)
        {
            highest_priority = pr[i];
            selected_process = i;
        }
    }
}

// No process available
if(selected_process == -1)
{
    current_time++;
}
else
{
    rt[selected_process]--;
    current_time++;

    if(rt[selected_process] == 0)
    {
        ct[selected_process] = current_time;

        tat[selected_process] =
            ct[selected_process] - at[selected_process];

        wt[selected_process] =
            tat[selected_process] - bt[selected_process];

        completed++;
    }
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",

```

```
        pid[i], at[i], bt[i], pr[i],  
        ct[i], tat[i], wt[i]);  
    }  
  
    return 0;  
}
```

riya gupta ,usn:1WA24CS235
Enter number of processes: 7

Process 1

Process ID: 6

Arrival Time:

6

Burst Time: 5

Priority: 6

Process 2

Process ID: 4

Arrival Time: 4

Burst Time: 1

Priority: 5

Process 3

Process ID: 2

Arrival Time: 1

Burst Time: 2

Priority: 4

Process 4

Process ID: 3

Arrival Time: 3

Burst Time: 4

Priority: 4

Process 5

Process ID: 1

Arrival Time: 0

Burst Time: 8

Priority: 3

Process 6

Process ID: 5

Arrival Time: 5

Burst Time: 6

Priority: 2

Process 7

Process ID: 7

Arrival Time: 10

Burst Time: 1

Priority: 1

PID	AT	BT	PR	CT	TAT	WT
6	6	5	6	27	21	16
4	4	1	5	22	18	17
2	1	2	4	17	16	14
3	3	4	4	21	18	14
1	0	8	3	15	15	7
5	5	6	2	12	7	1
7	10	1	1	11	1	0

=== Code Execution Successful ===

Round Robin CODE

```
#include <stdio.h>
```

```
int main() {
    printf("riya gupta ,usn:1WA24CS235 \n");
    int n, tq;
    int at[20], bt[20], rt[20];
    int wt[20], tat[20], rt_time[20];
    int time = 0, completed = 0;

    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(int i=0;i<n;i++){
        printf("Enter Arrival Time and Burst Time for P%d: ",i+1);
        scanf("%d %d",&at[i],&bt[i]);

        rt[i] = bt[i];
        rt_time[i] = -1;
    }

    printf("Enter Time Quantum: ");
    scanf("%d",&tq);

    while(completed < n){

        int executed = 0;

        for(int i=0;i<n;i++){

            if(at[i] <= time && rt[i] > 0){

                if(rt_time[i] == -1)
                    rt_time[i] = time - at[i];

                if(rt[i] > tq){
```

```

        time += tq;
        rt[i] -= tq;
    }
    else{
        time += rt[i];
        rt[i] = 0;

        tat[i] = time - at[i];
        wt[i] = tat[i] - bt[i];

        avg_wt += wt[i];
        avg_tat += tat[i];
        avg_rt += rt_time[i];

        completed++;
    }

    executed = 1;
}
}

if(executed == 0)
    time++;
}

printf("\nPID\tAT\tBT\tWT\tTAT\tRT\n");

for(int i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],wt[i],tat[i],rt_time[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);

return 0;
}

```

```
riya gupta ,usn:1WA24CS235
Enter number of processes: 5
Enter Arrival Time and Burst Time for P1: 0
5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 1
Enter Arrival Time and Burst Time for P4: 3 2
Enter Arrival Time and Burst Time for P5: 4 3
Enter Time Quantum: 2
```

PID	AT	BT	WT	TAT	RT
P1	0	5	9	14	0
P2	1	3	8	11	1
P3	2	1	2	3	2
P4	3	2	2	4	2
P5	4	3	6	9	3

```
Average Waiting Time = 5.40
Average Turnaround Time = 8.20
Average Response Time = 1.60
```

```
=== Code Execution Successful ===
```